Bachelor Thesis

April 7, 2026

Attribute–Embedding Correlation

A distribution divergence approach on Filtered Vector Search

Matteo Iulian Adam

of University of Zurich, Switzerland (23-744-519)

supervised by

Prof. Dr. Yannis Chronis, ETH
Prof. Dr. Dan Olteanu, UZH
Dr. Mageirakos Vasilis, ETH

University of Zurich[UZH]

Bachelor Thesis

Attribute–Embedding Correlation

A distribution divergence approach on Filtered Vector Search

Matteo Iulian Adam

University of Zurich[UZH]

Bachelor Thesis

Author: Matteo Iulian Adam, matteoiulian.adam@uzh.ch

URL: https://github.com/johndonneUZH/thesis

Project period: 2026-01-15 - 2026-07-01

Systems Group, Institute for Computing Platforms
Department of Computer Science (D-INFK), ETH Zurich

Acknowledgements

Abstract

Contents

1 Introduction **1**

1.1 Filtered Vector Search . . . 1

1.2 Attribute–embedding correlation . . . 1

1.3 Research Objectives & Questions . . . 1

1.4 Contributions . . . 1

1.5 Thesis Structure . . . 1

2 Background and related work **3**

2.1 Vector Search . . . 3

2.1.1 Vector representation . . . 3

2.1.2 Nearest Neighbor Search . . . 3

2.1.3 Approximate Nearest Neighbor Search . . . 3

2.1.4 Metrics for ANNS Evaluation . . . 4

2.2 Filtered Vector Search . . . 5

2.2.1 Selectivity . . . 5

2.2.2 Correlation . . . 6

2.3 Execution Strategies . . . 7

2.3.1 Pre-filtering . . . 7

2.3.2 Post-filtering . . . 8

2.3.3 Inline Filtering . . . 8

2.4 Research Gap . . . 8

3 Problem Definition **9**

3.1 Data Model . . . 9

3.2 Query Model . . . 9

3.3 Objective . . . 9

4 Dataset Construction **11**

4.1 Design Goals . . . 11

4.2 Correlated Attribute Generation . . . 11
4.2.1 Clustering . . . 11
4.2.2 Gaussian Weighting . . . 11
4.2.3 Soft Assignment . . . 11
4.2.4 Normalization & Quantization . . . 11
4.3 Uncorrelated Attribute Generation . . . 11
4.4 Query Range Generation . . . 11
4.4.1 Positive (correlated) . . . 11
4.4.2 Negative (anti-correlated) . . . 11
4.4.3 Random . . . 11
4.5 Selectivity Control . . . 11
4.6 Dataset Validation . . . 11
4.6.1 Neighbor consistency . . . 11
4.6.2 Cluster distributions . . . 11

5 Correlation Metric 13
5.1 Local vs Global Attribute Distributions . . . 13
5.2 Cluster-based Approximation . . . 13
5.3 Jensen–Shannon Divergence . . . 13
5.4 Lift (range alignment) . . . 13
5.5 Combined Score & Interpretation . . . 13

6 System & Execution Strategies 15
6.1 Pre-filtering . . . 15
6.2 Post-filtering . . . 15
6.3 Inline Filtering . . . 15
6.4 Implementation Details . . . 15

7 Experimental Setup 17
7.1 Offline computed statistics . . . 17
7.2 Evaluation Metrics . . . 17
7.3 Ground Truth Computation . . . 17

8 Results & Discussion 19

Bibliography **21**

A First Appendix **23**

B Second Appendix **25**

1. Introduction

1.1 Filtered Vector Search

1.2 Attribute–embedding correlation

1.3 Research Objectives & Questions

1.4 Contributions

1.5 Thesis Structure

2. Background and related work

2.1 Vector Search

2.1.1 Vector representation Modern data-intensive applications increasingly rely on representing complex objects (e.g., text, images or user behavior) as high-dimensional vectors. These vector representations, often referred to as embeddings, are typically learned using machine learning models and capture semantic or structural relationships in the data [1]. As a result, semantically similar objects are mapped to nearby points in a continuous vector space, allowing similarity between objects to be measured using geometric distance functions such as Euclidean distance or cosine similarity.

The widespread adoption of embeddings has led to the emergence of vector databases, which extend traditional database systems by enabling efficient similarity-based retrieval. Unlike conventional databases that operate primarily on structured attributes and exact-match queries, vector databases are designed to efficiently find similar items in large collections of embeddings by leveraging specialized index structures (e.g., graph-based or tree-based indexes) and approximate nearest neighbor search algorithms [2].

2.1.2 Nearest Neighbor Search Given a dataset of vectors $\{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, the nearest neighbor search problem consists of identifying the vectors that are closest to a query vector $q \in \mathbb{R}^d$ according to a distance (or dissimilarity) measure. In the k-nearest neighbor (kNN) setting, the goal is to retrieve the k closest vectors to q. This problem is also commonly referred to as the *post office problem*, where the objective is to find the closest facility to a given location.

The notion of "closeness" is defined through a distance function $D(\cdot, \cdot)$, which quantifies dissimilarity between vectors. Common choices include the Euclidean distance, cosine distance, and inner product-based measures. The choice of distance function plays a critical role, as it determines how similarity in the embedding space is interpreted and directly affects retrieval quality.

A straightforward solution is to compute the distance between the query and every vector in the dataset. While this exact approach guarantees optimal results, it has a time complexity of $\mathcal{O}(n \cdot d)$, which becomes prohibitive for large-scale datasets and high-dimensional spaces [3].

Nearest neighbor search is a fundamental problem in many domains, including information retrieval, machine learning, and computer vision [4]. However, as the dimensionality of the data increases, exact search methods suffer from the so-called *curse of dimensionality*, where distance computations become increasingly expensive and less discriminative.

These limitations motivate the use of approximate methods, which trade exactness for efficiency and enable scalable similarity search in high-dimensional spaces.

2.1.3 Approximate Nearest Neighbor Search To address the scalability limitations of exact nearest neighbor search, approximate nearest neighbor (ANN) methods have been developed. These methods trade exactness for efficiency by exploring only a subset of the dataset while aiming to preserve high recall. Instead of exhaustively comparing a query against all data points, ANN algorithms rely on specialized index structures and heuristics to efficiently identify promising candidates.

ANN methods can be broadly categorized into different families. A prominent class is based on hashing techniques, such as locality-sensitive hashing (LSH), which project high-dimensional vectors into lower-dimensional hash spaces in a way that preserves similarity [4]–[6]. The key idea behind LSH is to use hash functions that increase the probability of collision for similar vectors while reducing it for dissimilar ones. This allows candidate neighbors to be retrieved efficiently from hash buckets, enabling sub-linear query time. However, LSH-based methods may require multiple hash tables and can suffer from reduced accuracy or increased memory usage in very high-dimensional settings.

Graph-based indexing methods have demonstrated particularly strong empirical performance. In particular, the Hierarchical Navigable Small World (HNSW) graph organizes data points into a multi-layer structure that enables efficient navigation toward nearest neighbors through greedy search [7]. Due to its balance between efficiency and accuracy, HNSW has become a widely adopted method in modern vector search systems. To further improve scalability to very large datasets, including those stored on disk, systems such as DiskANN extend graph-based approaches by optimizing data layout and access patterns for disk-based storage [8]. These methods enable efficient nearest neighbor search over billion-scale datasets while maintaining high accuracy.

In addition to classical ANN structures, recent work has explored how data structures for nearest neighbor search can themselves be discovered or learned from data [9]. Similarly, learning-based indexing approaches have been proposed in which a model is trained to predict or guide the partitioning of the vector space so that search can be performed more efficiently [10]. These approaches illustrate that ANN performance depends not only on the search algorithm, but also on how the underlying search structure is constructed.

In this thesis, ANN search is implemented using the FAISS library, a widely used system for efficient similarity search over high-dimensional vectors [11]. FAISS provides a unified framework that implements a variety of ANN techniques, including graph-based methods such as HNSW, as well as exact search for ground truth computation. Its scalability and performance make it a standard choice for large-scale vector search applications.

2.1.4 Metrics for ANNS Evaluation Approximate nearest neighbor methods trade exactness for efficiency, making it necessary to evaluate both retrieval quality and computational performance. As a result, ANN systems are typically assessed along two main dimensions: retrieval accuracy and resource usage [11], [12].

A primary measure of retrieval quality is *recall@k*, which quantifies how many of the true nearest neighbors are retrieved by the approximate method. Let R_k^* denote the set of the k exact nearest neighbors and R_k the set returned by the ANN method. Then:

$$\text{recall@k} = \frac{|R_k \cap R_k^*|}{|R_k^*|}.$$

More generally, n-recall@k is defined as the fraction of the top-n ground truth nearest neighbors that appear in the top-k approximate results, where $n \leq k$ [11]. A common setting is $n = k$, in which case the measure corresponds to the intersection between the exact and approximate result sets. When $n = k = 1$, recall@1 reduces to a binary indicator of whether the nearest neighbor was correctly retrieved, and is therefore equivalent to accuracy. For $k > 1$, however, recall@k measures the fraction of correctly retrieved neighbors and should not be confused with accuracy in the general classification sense.

It is worth noting that the definition of recall@k is not entirely consistent across the literature. In some works, recall@k refers to 1-recall@k, while in others it corresponds to k-recall@k. In this thesis, we adopt the standard definition based on the fraction of correctly retrieved nearest neighbors.

In addition to retrieval accuracy, computational efficiency is a key metric. Query performance is typically measured in terms of latency (time per query) or throughput, often expressed as queries per second (QPS). These metrics capture the practical efficiency of ANN systems and are central to evaluating their scalability.

ANN methods also introduce additional resource considerations. These include memory usage for storing index structures, preprocessing or index construction time, and, in large-scale or disk-based systems, the number of input/output operations required during search [11]. For example, graph-based indexes require additional memory to store connectivity information, while compressed representations may reduce memory at the cost of reduced retrieval accuracy.

Overall, ANN evaluation involves analyzing trade-offs between retrieval accuracy and resource usage. Algorithms are typically tuned to achieve an optimal balance, where high recall is maintained while minimizing query time and resource consumption.

In this thesis, ANN methods are evaluated using recall@k to measure retrieval accuracy, and queries per second (QPS) and query latency to assess performance. Exact nearest neighbor search is used to compute ground truth results for comparison.

2.2 Filtered Vector Search

Filtered vector search extends nearest neighbor search by combining similarity-based retrieval in an embedding space with structured filtering over associated attributes. Given a query vector and a predicate over metadata, the goal is to retrieve the nearest neighbors that satisfy the predicate. This setting arises naturally in modern applications, where retrieval must simultaneously account for semantic similarity and structured constraints.

The difficulty arises because the filter does not simply reduce the number of candidates; it changes the effective search domain. For a predicate p, the query is evaluated over the subset $X_p \subseteq X$ of vectors whose records satisfy p, rather than over the full dataset. The geometry of X_p in the embedding space can differ significantly from that of X: it may be dense and well connected, or sparse and fragmented. Consequently, vectors satisfying the predicate need not lie near the query, even when they are the only admissible answers [13], [14].

This makes filtered vector search inherently query-dependent. Each predicate induces a different subset X_p, and therefore a different effective search space. Unlike standard nearest neighbor search, where a single index is built over a fixed distribution, filtered queries operate over many implicit subspaces defined at query time. These subspaces vary in size, density, and connectivity, which directly affects search behavior.

As a result, methods designed under assumptions about the global data distribution may behave inconsistently when applied to predicate-induced subsets. Existing literature reports inefficiencies, increased computational cost, and degraded retrieval quality when the filtered subset is poorly aligned with the structure of the embedding space [15], [16].

2.2.1 Selectivity

Selectivity is a fundamental concept in query processing that quantifies the fraction of the dataset satisfying a given predicate. Let X denote the dataset and p a predicate. The subset of elements satisfying p is defined as

$$X_p = \{x \in X \mid p(x) = \text{true}\}.$$

The selectivity of p is then given by

$$\text{sel}(p) = \frac{|X_p|}{|X|}.$$

Selectivity is closely related to cardinality, which refers to the absolute number of matching elements, i.e., $|X_p|$.

Cardinality estimation has been extensively studied in the database literature, as it plays a central role in query optimization [17], [18]. Classical approaches often rely on simplifying assumptions such as attribute independence, where the selectivity of a conjunction of predicates is approximated as the product of their individual selectivities. While this assumption simplifies estimation, it can lead to significant errors when attributes are correlated, as it ignores dependencies between them [18], [19]. Query planners rely on these estimates, together with other statistics such as histograms or sampling-based summaries, to choose efficient execution strategies. Inaccurate estimates can propagate through the planning process, leading to suboptimal decisions.

In filtered vector search, selectivity directly determines the size of the admissible search space. Given a query vector q, retrieval is performed over the restricted domain X_p rather than the full dataset X. When $\text{sel}(p)$ is high, the search space remains large and resembles the original dataset. When $\text{sel}(p)$ is low, the search is restricted to a small subset, which can significantly reduce the number of candidate vectors but also increase the difficulty of finding relevant neighbors. In particular, if $|X_p|$ is small, the nearest valid neighbors may lie far from the query in the embedding space, requiring more extensive exploration.

However, selectivity alone does not fully characterize query difficulty. Two predicates p_1 and p_2 with $\text{sel}(p_1) = \text{sel}(p_2)$ may induce subsets X_{p_1} and X_{p_2} with very different structures in the embedding space, leading to different retrieval behavior. In this thesis, we therefore evaluate queries across different selectivity levels while focusing on the effect of attribute–embedding correlation.

2.2.2 Correlation As discussed above, selectivity alone does not fully characterize the difficulty of filtered vector search. In addition to the size of the admissible set X_p, performance also depends on how the predicate-induced subset is arranged in the embedding space relative to the query.

Let

$$X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$$

be the set of embeddings, let p be a predicate, and let

$$X_p = \{x \in X \mid p(x) = \text{true}\}$$

denote the subset of vectors satisfying p. For a query vector $q \in \mathbb{R}^d$, filtered vector search seeks the k nearest neighbors of q within X_p.

To formalize the interaction between the predicate and the embedding space, define

$$g_k(q, S) = \frac{1}{k} \sum_{x \in \text{NN}_k(q,S)} d(q, x),$$

where $\text{NN}_k(q, S)$ denotes the k nearest neighbors of q in a set S, and d is the distance function used for retrieval. Thus, $g_k(q, X_p)$ is the average distance from q to its k nearest admissible neighbors under predicate p [13].

To determine whether these admissible neighbors are unusually close or unusually far, we compare them to a baseline in which the predicate has no geometric structure in the embedding space. Let

$$R_p \subseteq X$$

be a random subset of size $|X_p|$, sampled uniformly from X. Then R_p represents a hypothetical subset with the same cardinality as X_p, but without any predicate-specific spatial organization.

We define the query-level correlation of (q, p) as

$$C(q, p) = \mathbb{E}[g_k(q, R_p)] - g_k(q, X_p),$$

where the expectation is taken over the random draw of R_p.

This quantity admits a natural interpretation:

$$C(q,p) > 0 \quad \Longrightarrow \quad g_k(q, X_p) < \mathbb{E}[g_k(q, R_p)],$$

meaning that the admissible neighbors in X_p are, on average, closer to the query than would be expected under a random subset of the same size. In this case, the query exhibits *positive correlation*.

Conversely,

$$C(q,p) < 0 \quad \Longrightarrow \quad g_k(q, X_p) > \mathbb{E}[g_k(q, R_p)],$$

meaning that the admissible neighbors are farther from the query than expected under the random baseline. In this case, the query exhibits *negative correlation*.

Finally,

$$C(q,p) \approx 0$$

indicates *no correlation*, i.e., the predicate-induced subset behaves, from the perspective of the query, similarly to a random subset of the same size.

Intuitively, positive correlation means that the predicate selects vectors that are geometrically aligned with the query region: the valid answers tend to lie inside or near the query's local neighborhood. Negative correlation means the opposite: although vectors satisfying the predicate exist, they are systematically displaced from the query's neighborhood, so retrieving them requires exploration beyond the region that would be most promising under vector similarity alone.

This distinction is fundamental for filtered vector search. Two predicates may have the same selectivity, i.e.,

$$|X_{p_1}| = |X_{p_2}|,$$

while inducing very different values of $C(q,p)$. In that case, the two queries have the same number of admissible vectors but differ substantially in geometric difficulty. Correlation therefore captures an aspect of filtered vector search that selectivity alone cannot express.

Correlation therefore captures a fundamental aspect of filtered vector search that selectivity alone cannot express. In this thesis, we develop a practical method to estimate this quantity efficiently.

2.3 Execution Strategies

Execution strategies for filtered vector search define how structured filter predicates are combined with nearest neighbor search.

The main difference between execution strategies is when and how the filter is applied in relation to the search process. This design choice directly affects performance, efficiency, and result quality.

Formally, given a dataset $D \subset \mathbb{R}^d$ and a Boolean filter predicate ϕ, the goal is to compute the top-k nearest neighbors of a query vector $q \in \mathbb{R}^d$ restricted to the subset

$$D_\phi = \{x \in D \mid \phi(x) = \text{true}\}.$$

In this thesis, we focus on three main execution strategies.

2.3.1 Pre-filtering

In pre-filtering, the filter predicate is evaluated first, producing the subset D_ϕ. The nearest neighbor search is then executed exclusively on this subset.

This approach guarantees that all returned results satisfy the filter and directly focuses the search on the relevant candidates. However, most vector indexes are designed for the full dataset, so we cannot use them for dynamically restricted subsets. As a result, their effectiveness may degrade, particularly for filters which induce a big subset (high selectivity), since we have to compute NN search on many vectors, and that's very expensive with no indexes.

2.3.2 Post-filtering In post-filtering, the nearest neighbor search is performed over the entire dataset without considering the filter predicate. The filter is applied only after retrieving a set of candidate neighbors.

This allows the index to operate under its optimal conditions. However, it does not guarantee that k valid results are returned. If many retrieved neighbors do not satisfy the filter, additional candidates must be examined, increasing query cost and potentially affecting efficiency.

2.3.3 Inline Filtering Inline filtering incorporates the filter predicate directly into the search procedure. During index traversal, candidates that do not satisfy the filter are pruned, and only valid vectors are considered for distance evaluation and result construction.

This approach avoids materializing the filtered subset (as in pre-filtering) and reduces wasted work on invalid candidates (as in post-filtering). However, it requires index structures and search algorithms that support predicate-aware traversal, which increases implementation complexity.

2.4 Research Gap Filtered vector search systems often exploit dependencies within the data to improve performance, for example in selectivity estimation [20], index construction, or query execution [13]. In these settings, such relationships play an important role in guiding system behavior and improving efficiency.

In this thesis, we adopt a more general and systematic perspective on modeling these dependencies. Rather than incorporating them implicitly within a specific method, we treat them as a standalone property of the data and derive a quantitative measure based on database statistics. This measure captures the interaction between filter predicates and the underlying vector distribution, enabling a more principled analysis of how filtering affects nearest neighbor search.

A key aspect of our approach is that the proposed formulation is independent of any particular index structure or execution strategy. It can therefore be used as a generic component and integrated into existing systems without requiring substantial modifications. This makes it applicable across a wide range of filtered vector search settings, including both exact and approximate methods.

By introducing a reusable and system-agnostic characterization of these dependencies, this thesis facilitates the development of methods that can incorporate such information in a modular way. Rather than targeting a single specialized solution, we provide a general abstraction that can be leveraged to improve different aspects of filtered vector search, including query planning, candidate selection, and execution strategy design.

3. Problem Definition

compute and verify the new proposed metric introduced in 5.5

3.1 Data Model - handcrafted dataset (proof of concept) = sift1m + generated attributes - 1M vectors w/ 128D - glove

3.2 Query Model - 10000 vectors w/ 128D - recall@k w/ k = 100

3.3 Objective

4. Dataset Construction

4.1 Design Goals

4.2 Correlated Attribute Generation

4.2.1 Clustering

4.2.2 Gaussian Weighting

4.2.3 Soft Assignment

4.2.4 Normalization & Quantization

4.3 Uncorrelated Attribute Generation

4.4 Query Range Generation

4.4.1 Positive (correlated)

4.4.2 Negative (anti-correlated)

4.4.3 Random

4.5 Selectivity Control

4.6 Dataset Validation

4.6.1 Neighbor consistency

4.6.2 Cluster distributions

5. Correlation Metric

5.1 Local vs Global Attribute Distributions

5.2 Cluster-based Approximation

5.3 Jensen–Shannon Divergence

5.4 Lift (range alignment)

5.5 Combined Score & Interpretation

6. System & Execution Strategies

6.1 Pre-filtering

6.2 Post-filtering

6.3 Inline Filtering

6.4 Implementation Details

7. Experimental Setup

7.1 Offline computed statistics

7.2 Evaluation Metrics

7.3 Ground Truth Computation

8. Results & Discussion

Bibliography

[1] F. Almeida and G. Xexéo, *Word embeddings: A survey*, 2023. arXiv: 1901.09069 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1901.09069.

[2] L. Ma, R. Zhang, Y. Han, *et al.*, *A comprehensive survey on vector database: Storage and retrieval technique, challenge*, 2026. arXiv: 2310.11703 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2310.11703.

[3] S. Arya, D. M. Mount, N. S. Netanyahu, R. Silverman, and A. Y. Wu, "An optimal algorithm for approximate nearest neighbor searching fixed dimensions," *J. ACM*, vol. 45, no. 6, 891–923, Nov. 1998, ISSN: 0004-5411. DOI: 10.1145/293347.293348. [Online]. Available: https://doi.org/10.1145/293347.293348.

[4] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98, Dallas, Texas, USA: Association for Computing Machinery, 1998, 604–613, ISBN: 0897919629. DOI: 10.1145/276698.276876. [Online]. Available: https://doi.org/10.1145/276698.276876.

[5] O. Jafari, P. Maurya, P. Nagarkar, K. M. Islam, and C. Crushev, *A survey on locality sensitive hashing algorithms and their applications*, 2021. arXiv: 2102.08942 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2102.08942.

[6] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, Edinburgh, Scotland, 1999, pp. 518–529. [Online]. Available: https://www.cs.princeton.edu/courses/archive/spring13/cos598C/Gionis.pdf.

[7] Y. A. Malkov and D. A. Yashunin, *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*, 2018. arXiv: 1603.09320 [cs.DS]. [Online]. Available: https://arxiv.org/abs/1603.09320.

[8] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32, Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf.

[9] O. Salemohamed, L. Charlin, S. Garg, V. Sharan, and G. Valiant, *Discovering data structures: Nearest neighbor search and beyond*, 2024. arXiv: 2411.03253 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2411.03253.

[10] C.-Y. Chiu, A. Prayoonwong, and Y.-C. Liao, *Learning to index for nearest neighbor search*, 2019. arXiv: 1807.02962 [cs.IR]. [Online]. Available: https://arxiv.org/abs/1807.02962.

[11] M. Douze, A. Guzhva, C. Deng, *et al.*, *The faiss library*, 2025. arXiv: 2401.08281 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2401.08281.

[12] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with gpus*, 2017. arXiv: 1702.08734 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1702.08734.

[13] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, *Acorn: Performant and predicate-agnostic search over vector embeddings and structured data*, 2024. arXiv: 2403.04871 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2403.04871.

[14] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, Dec. 2024. DOI: 10.1145/3698822. [Online]. Available: https://doi.org/10.1145/3698822.

[15] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, *Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints*, 2022. arXiv: 2203.13601 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2203.13601.

[16] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportunities," *Proc. VLDB Endow.*, vol. 18, no. 12, 5488–5492, Aug. 2025, ISSN: 2150-8097. DOI: 10.14778/3750601.3750700. [Online]. Available: https://doi.org/10.14778/3750601.3750700.

[17] C. A. Lynch, "Selectivity estimation and query optimization in large databases with highly skewed distribution of column values," in *Very Large Data Bases Conference*, 1988. [Online]. Available: https://api.semanticscholar.org/CorpusID:20418479.

[18] H. Zhang, C. Mayer, M. A. Khamis, D. Olteanu, and D. Suciu, *Lpbound: Pessimistic cardinality estimation using ℓ_p-norms of degree sequences*, 2025. arXiv: 2502.05912 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2502.05912.

[19] L. Getoor, B. Taskar, and D. Koller, "Selectivity estimation using probabilistic models," *SIGMOD Rec.*, vol. 30, no. 2, 461–472, May 2001, ISSN: 0163-5808. DOI: 10.1145/376284.375727. [Online]. Available: https://doi.org/10.1145/376284.375727.

[20] C. Mayer, H. Zhang, M. Abo Khamis, K. Deeds, D. Olteanu, and D. Suciu, "Corrbound: Cardinality estimation accounting for inter- and intra-relation correlations," in *Proceedings of the ACM SIGMOD Conference*, to appear, 2026.

A. First Appendix

B. Second Appendix